

Machine Learning

Lecture 3

Simple Classifiers: Nearest Centroids

Felix Bießmann

Berliner Hochschule für Technik & Einstein Center for Digital Future

Overview of today's lecture

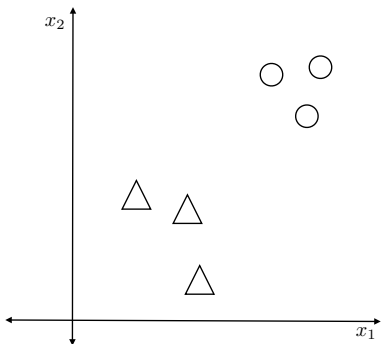
Today we will introduce a simple multi-class classifier:
The **Nearest Centroid Classifier**

- Simple to implement
- Fast and memory efficient
- Interpretable

Prototypes: Psychological Models of Abstract Ideas

- Neurons receive non-linearly filtered sensory input $\mathbf{x}$
- How can abstract concepts be learned from this information?

Prototypes: Psychological Models of Abstract Ideas



How do humans categorize?

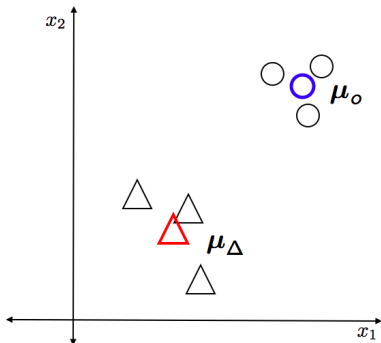
Psychologists postulated that we learn **prototypes** [Jaekel, 2007; Posner and Keele, 1968]

Toy data example:

Two dimensional input $\mathbf{x} \in \mathbb{R}^2$

Two *classes* of data, Δ and $\circ$

Prototypes: Psychological Models of Abstract Ideas



How to compute prototypes?

Simplest solution: the class means μ_Δ and μ_o

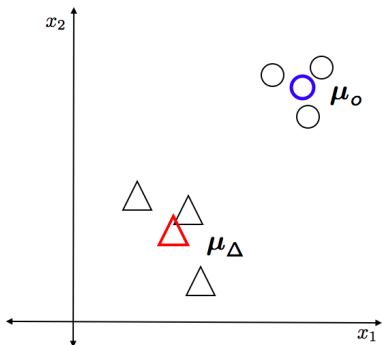
$$\mu_\Delta = 1/N_\Delta \sum_n^{N_\Delta} \mathbf{x}_{\Delta,n}$$

$$\mu_o = 1/N_o \sum_n^{N_o} \mathbf{x}_{o,n}$$

Distance from μ_Δ to new data $\mathbf{x}$

$$\|\mu_\Delta - \mathbf{x}\|_2$$

Prototypes: Psychological Models of Abstract Ideas



For new data $\mathbf{x}$ check:
Is $\mathbf{x}$ more similar to μ_o ?

$$\|\mu_{\Delta} - \mathbf{x}\| > \|\mu_o - \mathbf{x}\|$$

yes? $\rightarrow \mathbf{x}$ belongs to μ_o

no? $\rightarrow \mathbf{x}$ belongs to μ_{Δ}

This is called a
nearest centroid classifier

Nearest Centroid Classification

This algorithm is formalized in the following pseudocode:

Algorithm 1 Nearest Centroid Prediction

Require: Data point $\mathbf{x} \in \mathbb{R}^D$, class centroids $\boldsymbol{\mu}_k$, $k \in \{1, \dots, K\}$

Ensure: Class membership k^*

- 1: # Compute nearest class centroid
 - 2: $k^* = \operatorname{argmin}_k \|\boldsymbol{\mu}_k - \mathbf{x}\|_2$.
-

Computing Nearest Centroids

- Here we define prototypes as the **class-centroids**
- Class centroids are just the mean of each class
- Means can be computed in a *batch* or *streaming* fashion

Batch Computations vs. Streaming

Solutions for algorithms can be obtained

- In Batch Mode:
 - Use all available data at once
 - Requires to store all data in memory
- In Streaming Mode:
 - Use one data point at a time
 - Requires to store **only** centroids

Nearest Centroid Classification Algorithm (Batch Mode)

Algorithm 2 Computation of Class-Centroids

Require: data $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^D$, labels $y_1, \dots, y_N \in \{1, \dots, K\}$

Ensure: Class means $\boldsymbol{\mu}_k$, $k \in \{1, \dots, K\}$

- 1: # Initialize means and counters for each class
 - 2: # Computation of class means
 - 3: **for** Class $k = 1, \dots, K$ **do**
 - 4: $\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{i=1}^{N_k} \mathbf{x}_i$
 - 5: **end for**
-

Iterative Computation of the Mean

Given the mean μ_{N-1} computed from $N - 1$ samples we want to update μ_{N-1} with the N th sample $\mathbf{x}_N$ to obtain μ_N

$$\begin{aligned}\mu_N &= \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n \\ &= \frac{1}{N} \sum_{n=1}^{N-1} \mathbf{x}_n + \frac{1}{N} \mathbf{x}_N \\ &= \frac{N-1}{N} \underbrace{\frac{1}{N-1} \sum_{n=1}^{N-1} \mathbf{x}_n}_{\mu_{N-1}} + \frac{1}{N} \mathbf{x}_N \\ &= \frac{N-1}{N} \mu_{N-1} + \frac{1}{N} \mathbf{x}_N\end{aligned}$$

Nearest Centroid Classification Algorithm (Streaming)

Algorithm 3 Iterative computation of Class-Centroids

Require: data $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^D$, labels $y_1, \dots, y_N \in \{1, \dots, K\}$

Ensure: Class means μ_k , $k \in \{1, \dots, K\}$

1: # Initialize means and counters for each class

2: $\forall k : \mu_k = \mathbf{1} \cdot 0, N_k = 0$

3: # Iterative computation of class means

4: **for** Data point $i = 1, \dots, N$ **do**

5: # Update means and counters

6: $k = y_i$

7: $\mu_k = \frac{N_k}{N_k+1} \mu_k + \frac{1}{N_k+1} \mathbf{x}_i$

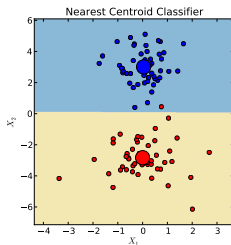
8: $N_k = N_k + 1$

9: **end for**

When Should I Use NCC?

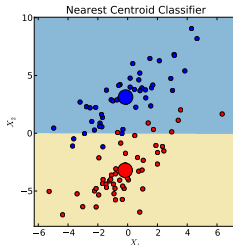
Consider a two dimensional data set

Uncorrelated features:



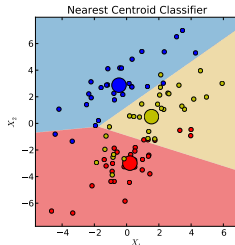
NCC works

Correlated features



NCC fails

Also with 3 Classes



NCC fails

Nearest Centroid Classification and Linear Classification

- NCC works well for uncorrelated data
 - Decision boundary of NCC is a line in feature space
- NCC is a *Linear Classifier*
- We now show that mathematically
- Question: What is the line that separates the classes?

From Prototypes to Linear Classification

Data point $\mathbf{x}$ is assigned to class $\circ$
if its distance to $\mu_{\circ}$ is smaller than to μ_{Δ} :

$$\begin{aligned} \text{distance}(\mathbf{x}, \mu_{\Delta}) &> \text{distance}(\mathbf{x}, \mu_{\circ}) \\ \|\mathbf{x} - \mu_{\Delta}\|_2 &> \|\mathbf{x} - \mu_{\circ}\|_2 \end{aligned} \tag{1}$$

From Prototypes to Linear Classification

$$\text{distance}(\mathbf{x}, \mu_{\Delta}) > \text{distance}(\mathbf{x}, \mu_o) \quad (1)$$

$$\|\mathbf{x} - \mu_{\Delta}\|_2 > \|\mathbf{x} - \mu_o\|_2$$

$$\Leftrightarrow \|\mathbf{x} - \mu_{\Delta}\|_2^2 > \|\mathbf{x} - \mu_o\|_2^2$$

$$\Leftrightarrow \mathbf{x}^T \mathbf{x} - 2\mu_{\Delta}^T \mathbf{x} + \mu_{\Delta}^T \mu_{\Delta} > \mathbf{x}^T \mathbf{x} - 2\mu_o^T \mathbf{x} + \mu_o^T \mu_o$$

$$\Leftrightarrow \mu_{\Delta}^T \mathbf{x} - \mu_{\Delta}^2/2 < \mu_o^T \mathbf{x} - \mu_o^2/2$$

$$\Leftrightarrow 0 < \underbrace{(\mu_o - \mu_{\Delta})^T \mathbf{x}}_{\mathbf{w}} - \underbrace{1/2(\mu_o^T \mu_o - \mu_{\Delta}^T \mu_{\Delta})}_{\beta}$$

From Prototypes to Linear Classification

$$\text{distance}(\mathbf{x}, \mu_{\Delta}) > \text{distance}(\mathbf{x}, \mu_o) \quad (1)$$

$$\|\mathbf{x} - \mu_{\Delta}\|_2 > \|\mathbf{x} - \mu_o\|_2$$

$$\Leftrightarrow \|\mathbf{x} - \mu_{\Delta}\|_2^2 > \|\mathbf{x} - \mu_o\|_2^2$$

$$\Leftrightarrow \mathbf{x}^T \mathbf{x} - 2\mu_{\Delta}^T \mathbf{x} + \mu_{\Delta}^T \mu_{\Delta} > \mathbf{x}^T \mathbf{x} - 2\mu_o^T \mathbf{x} + \mu_o^T \mu_o$$

$$\Leftrightarrow \mu_{\Delta}^T \mathbf{x} - \mu_{\Delta}^2/2 < \mu_o^T \mathbf{x} - \mu_o^2/2$$

$$\Leftrightarrow 0 < \underbrace{(\mu_o - \mu_{\Delta})^T \mathbf{x}}_{\mathbf{w}} - \underbrace{1/2(\mu_o^T \mu_o - \mu_{\Delta}^T \mu_{\Delta})}_{\beta}$$

Linear Classification

$$\mathbf{w}^T \mathbf{x} - \beta = \begin{cases} > 0 & \text{if } \mathbf{x} \text{ belongs to class } o \\ < 0 & \text{if } \mathbf{x} \text{ belongs to class } \Delta \end{cases} \quad (2)$$

Linear Classification

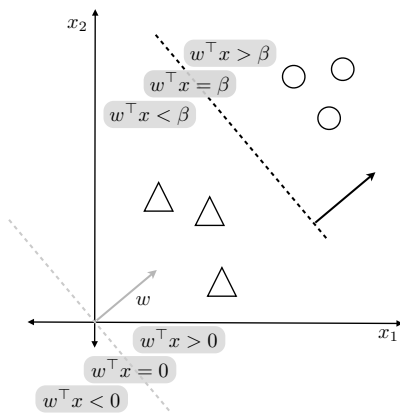
- A linear classifier predicts class for data point $\mathbf{x}$ as

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} - \beta \quad (3)$$

- $\mathbf{w}$ defines the *decision boundary*
- β defines the *decision threshold*
- For NCC, $\mathbf{w}$ is the difference of the class means

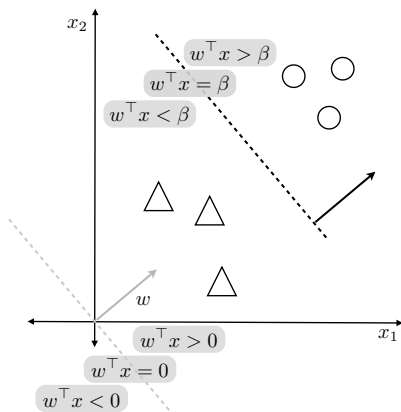
$$\mathbf{w}_{\text{NCC}} = \mu_o - \mu_\Delta \quad (4)$$

Linear Classification



$$\mathbf{w}^\top \mathbf{x} - \beta = \begin{cases} > 0 & \text{if } \mathbf{x} \text{ belongs to } o \\ < 0 & \text{if } \mathbf{x} \text{ belongs to } \Delta \end{cases}$$

Linear Classification



$$\mathbf{w}^\top \mathbf{x} - \beta = \begin{cases} > 0 & \text{if } \mathbf{x} \text{ belongs to } o \\ < 0 & \text{if } \mathbf{x} \text{ belongs to } \Delta \end{cases}$$

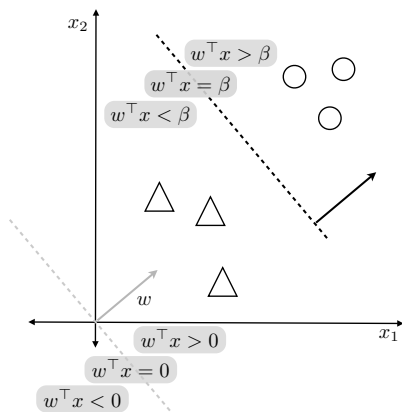
The *threshold* β can be included in $\mathbf{w}$

$$\tilde{\mathbf{x}} \leftarrow \begin{bmatrix} 1 \\ \mathbf{x} \end{bmatrix} \quad \tilde{\mathbf{w}} \leftarrow \begin{bmatrix} -\beta \\ \mathbf{w} \end{bmatrix}$$

such that

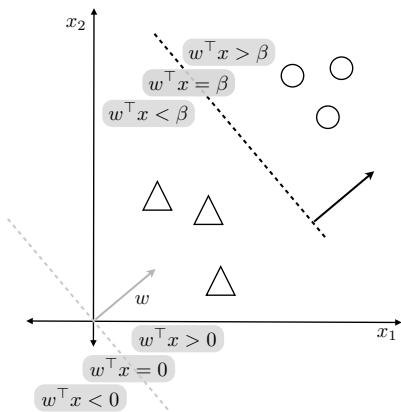
$$\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} = \mathbf{w}^\top \mathbf{x} - \beta.$$

Linear Classification



- What is a good $\mathbf{w}$?
- Some proposals:
 - Logistic Regression
 - Perceptrons
 - Support Vector Machines
 - Ridge Regression
- Linear methods have different ways of defining what a good $\mathbf{w}$ is

Linear Classification



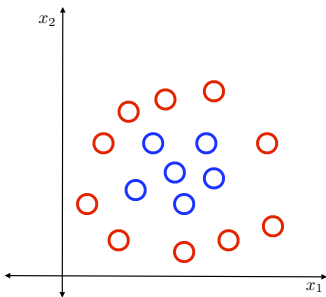
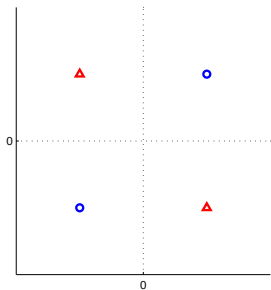
Nearest Centroid Classification is a simple linear classifier

$$w = \mu_o - \mu_{\Delta} \quad (5)$$

$$\beta = -1/2(\mu_o^\top \mu_o - \mu_{\Delta}^\top \mu_{\Delta})$$

Problems with Linear Classification

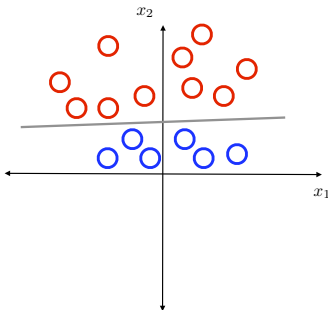
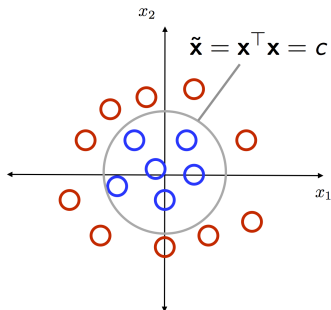
Linear Classifiers fail on non-linear problems:



Try to separate these two-class data sets with **a single line**

Problems with Linear Classification

If we can model the non-linearity,
we can create new features $\tilde{\mathbf{x}}$ which are linearly separable



But what if we do not know the non-linearity?

Summary

Psychologists postulated we learn **Prototypes**

- Prototypes can be the class means

- Prototype theory is closely related to linear classification

Nearest Centroid Classification

- New data is assigned to class with closest centroid

- Memory efficient: only requires to store D -dimensional centroids

- Iterative/Streaming version can scale to large data sets

- Nearest Centroid Classifiers assume uncorrelated features

Linear Classification

- Nearest Centroid Classifiers are Linear Classifiers

- Linear Classifiers fail on nonlinear classification problems

References

F. Jaekel. *Some Theoretical Aspects of Human Categorization Behaviour: Similarity and Generalization*. PhD thesis, 2007.

M. I. Posner and S. W. Keele. On the genesis of abstract ideas. *Journal of Experimental Psychology*, 77(3):353–363, 1968.